

MAKERERE UNIVERSITY

COLLEGE OF COMPUTING AND INFORMATION SCIENCES

SCHOOL OF COMPUTING AND INFORMATICS TECHNOLOGY

Department of Computer Science

CSC 1102: Structured and Object-Oriented Programming

Java Exam Project (Student Management System)

. **GitHub repository Link**: https://github.com/akatuhaderek-code/Student-Management-System.git

GROUP 16

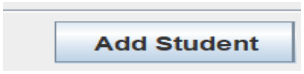
NAME	STUDENT NO.	REG NO.
Naggayi Hellenah	2600700311	26/U/0311
Namubiru Clarissa Magezi	2600703610	26/U/03610/PS
Nakayiwa Patience Mubeezi	2600700312	26/U/0312
Mwesigwa Bbosa Cirsten	2600730071	26/U/30071/PS
Ankunda Daniella	2600703487	26/U/03487 PSA
Akatuha Derick	2600700299	26/U/0299

1. PROJECT DESCRIPTION

Our group developed a simple Student Management System application using Java. The application features a graphical user interface (GUI) built with the Java Swing framework. It allows an administrator to easily register new students, search for existing student records, and delete records directly through an interactive table grid. The application stores data locally in a standard array without needing an external database

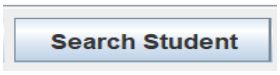
2. MAIN FEATURES AND THEIR APPEARANCE IN THE GUI

• **Add Student**:



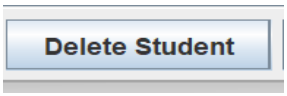
Validates user input to ensure fields are not left blank and that the age is a positive number. It checks if the Student ID already exists or if the storage list is full, showing a specific error message for each scenario.

• **Search Student**:



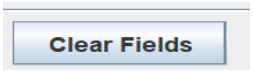
Searches through the records by entering a Student ID. When found, it displays the student's details in a message popup.

• **Delete Student**:



Allows the user to click and highlight a student's row in the table, then press the "Delete Student" button to completely remove them from the system.

• **Clear Fields**:



A simple helper button that resets all input text fields instantly.

3. LIST OF CLASSES AND THEIR INTERACTIONS

Our application is split into three main classes that work together sequentially:

• **Student class (Data Model)**: This class is the basic blueprint for a student object. It stores the details (ID, Name, Course, Age) using private variables and provides public getter methods. Interaction: It does not call any other class. Instead, it is instantiated by the GUI and stored inside the Management array.

• **StudentManagement class(Back-end Logic)**: This class acts as the local database. It declares a fixed-size array (Student[] students = new Student[100]) and a count variable to track the active number of records. Interaction: It receives Student objects from the GUI to add them to the array. It also receives text strings (IDs) from the GUI to perform searches or run loop shifts during deletions. It passes back status integers (1, -1, -2) or Student objects to the GUI.

• **StudentGUI class (Front end User Interface)**: This class sets up the visual window grid and buttons using Java Swing. It creates a single instance of StudentManagement (manager) at the top. Interaction: When a user clicks a button, the GUI collects the text input, creates a new Student object, and passes it to

the manager instance. It takes the response code back from the manager to decide whether to add a visual row to the JTable or display an error dialog box.

Figure 1 The clean GUI appears as below

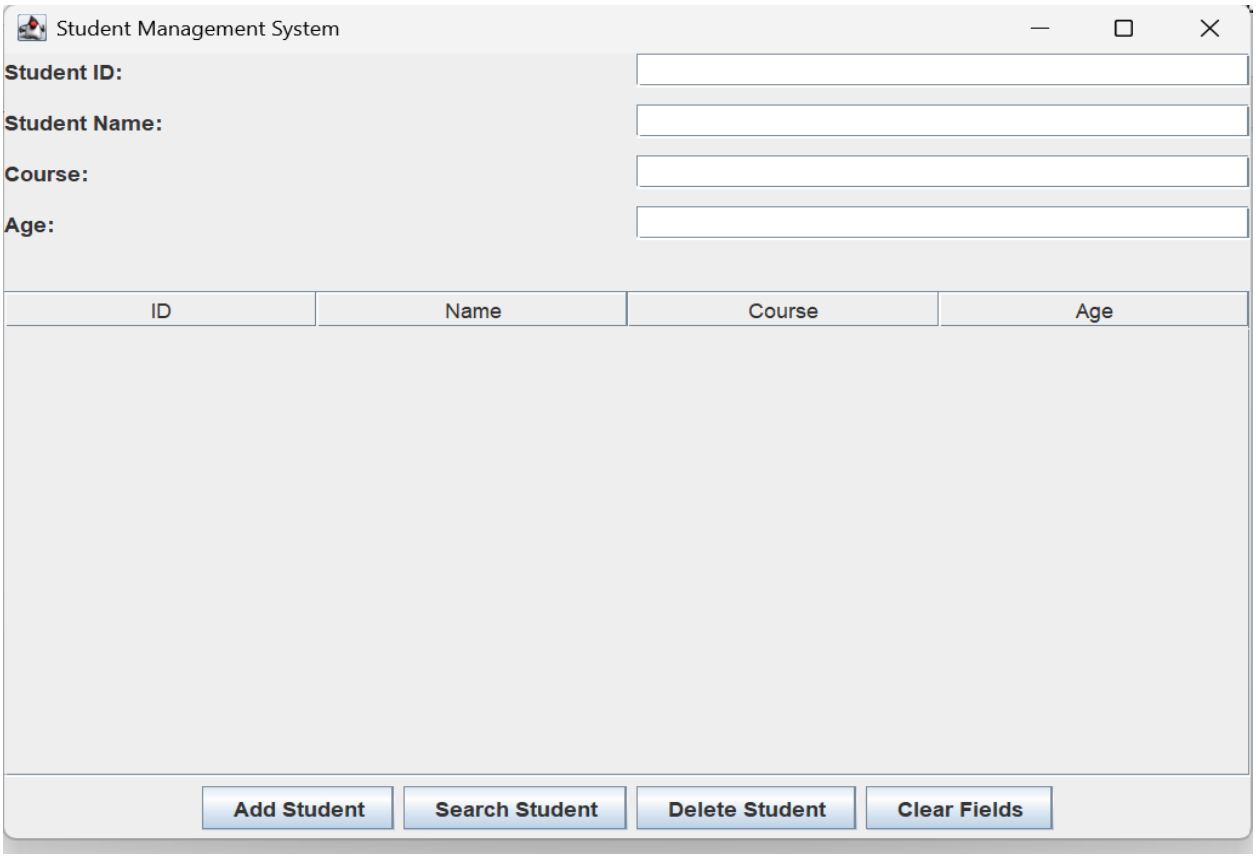


Figure 2 Shows adding an incomplete file does not work , you have to fill all fields

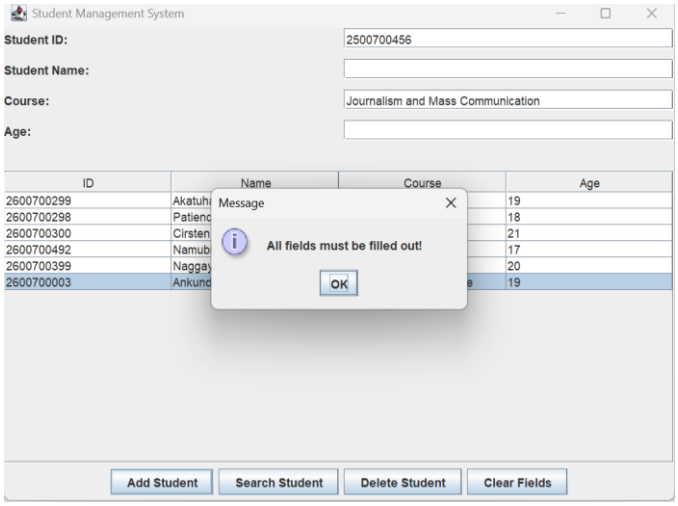


Figure 3 Shows a complete file added successfully

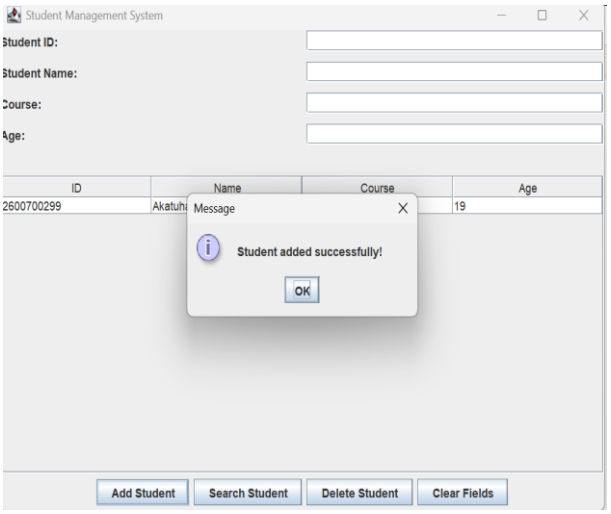


Figure 4 Trying to look for a student , just type in the ID and search and you find your student with ease. For example we searched for Bbosa J by typing in his ID and clicked search as shown below

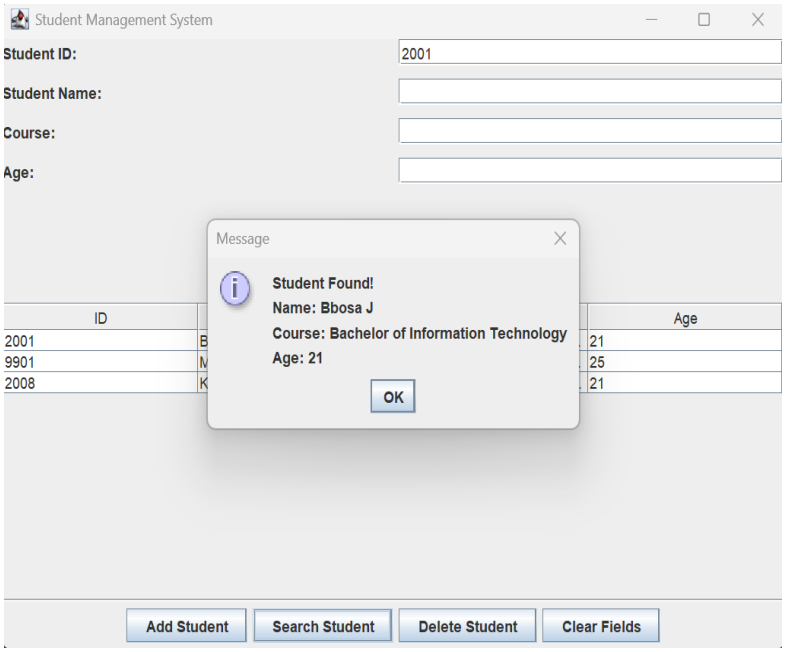


Figure 5 Select the row to delete and click delete action button

